

SYNTHETIX · SECURITY REVIEW

Synthetix

Treasury Updates.

A security review of Treasury Updates.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Synthetix	Treasury Updates	Guardian	April 7 – April 9, 2025	April 17, 2025

§ 00

Contents.

28 pages — overview, confidence ranking, scope, 15 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 15 findings	P. 09
§ 05	Disclaimer	P. 27
§ 06	About Guardian	P. 28

§ 01

Overview.

Guardian reviewed Treasury Updates across a single review round. The final report includes 2 High, 2 Medium, 11 Low findings.

AUDIT FIRM	REPOSITORY
Guardian	Synthetixio/synthetix-v3
CLIENT	METHODOLOGY
Synthetix	Smart contract · Manual · Main Review
PROJECT	MAIN REVIEW COMMIT
Treasury Updates	0c32d0cfc8a8
PREPARED BY	
Guardian	
AUDIT WINDOW	
April 7 – April 9, 2025	
FINAL REPORT	
April 17, 2025	

1.1 Confidence Ranking

4 / 5

High Confidence

Guardian assigns a Confidence Ranking of 4 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
 Critical	0	0	0	0	0
 High	2	0	2	0	0
 Medium	2	0	2	0	0
 Low	11	0	11	0	0
 Informational	0	0	0	0	0
All severities	15	0	15	0	0

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence ASSIGNED	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Treasury Updates in `Synthetixio/synthetix-v3`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	Anyone can loop saddle-unsaddle and drain depositRewards	Logical Error	HIGH	ACKNOWLEDGED	p. 9
H-02	Liquidated collateral is credited twice	Unexpected Behavior	HIGH	ACKNOWLEDGED	p. 11
M-01	burnUsd locks the available collateral	Rewards	MEDIUM	ACKNOWLEDGED	p. 13
M-02	Reward token removal can be abused for potential future earnings	Rewards	MEDIUM	ACKNOWLEDGED	p. 15
L-01	Delegation in liquidateToTreasury lacks checks	Validation	LOW	ACKNOWLEDGED	p. 16
L-02	Anyone can DoS setDepositRewardConfigurations	Informational	LOW	ACKNOWLEDGED	p. 17
L-03	Unsaddle may be blocked because of rewards	DoS	LOW	ACKNOWLEDGED	p. 18
L-04	The oracleManager should properly adjust decimals	Math	LOW	ACKNOWLEDGED	p. 19
L-05	Unclaimed rewards are likely lost when DepositRewardToken is removed	Rewards	LOW	ACKNOWLEDGED	p. 20
L-06	Precision Revert in withdrawMarketCollateral During Unsaddle	Logical Error	LOW	ACKNOWLEDGED	p. 21
L-07	Approvals are not reset	Logical Error	LOW	ACKNOWLEDGED	p. 22

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-08	The treasuryAccountId is not checked for existence when liquidating	Validation	LOW	ACKNOWLEDGED	p. 23
L-09	The pool loaded in liquidateToTreasury is not checked for existence	Validation	LOW	ACKNOWLEDGED	p. 24
L-10	_decimalPow() supports only 18 decimals precision base	Math	LOW	ACKNOWLEDGED	p. 25
L-11	Misplaced Comment in LiquidationModule.sol	Best Practices	LOW	ACKNOWLEDGED	p. 26

HIGH **H-01**

ACKNOWLEDGED

Anyone can loop saddle-unsaddle and drain depositRewards

CATEGORY Logical Error
LOCATION TreasuryMarket
REVIEW Main Review

DESCRIPTION

In the Treasury Market, the `saddle()` function either allocates rewards or initiates debt repayment based on the user's account debt:

- Zero debt: Rewards are allocated to `depositRewards[accountId][config.token]`.
- Non-zero debt: Debt repayment is prioritized.

Only one of these actions occurs per `saddle()` call.

Rewards are recorded during `saddle()` but not cleared during `unsaddle()`, relying on the next `saddle()` to overwrite or reset them. This assumption introduces vulnerabilities.

Issue 1: Reward Persistence Across Debt StatesSteps:

1. User saddles with zero debt, earning rewards in `depositRewards`.
2. User unsaddles post vesting, claims rewards, but `depositRewards` remains uncleared.
3. User saddles again with non-zero debt (e.g., accrued via delegation to other pools).
4. Since `debt > 0`, `saddle()` prioritizes repayment and does not reset prior `depositRewards`.
5. User loops this process of saddling with non zero debt and unsaddle.

This could have been critical, however since the minimum delegation time is expected to be 7 days, the damage could be capped to first loop itself.

We tested this on a mainnet fork with `poolId = 1` and confirmed that delegation is still possible for other pools, allowing the scenario described above.

Synthetix also handles cases where a user could saddle with a debt after their first saddle [here] (<https://github.com/Synthetixio/synthetix-v3/blob/0c32d0cfc8a8cc7f22d368383fa10c598a318311/markets/treasury-market/contracts/TreasuryMarket.sol#L169-L170>), which enables this attack vector.

H-01 Anyone can loop saddle-unsaddle and drain depositRewards

CONTINUED

Issue 2: Reward Leakage via Config Manipulation

Even if Synthetix restricts debt accrual in other pools post-Treasury Market migration, reward leakage persists.

Steps:

1. User saddles while Token A is in `depositRewardConfigurations`, accruing rewards.
2. User unsaddles, but `depositRewards` for Token A is not reset.
3. SNX removes Token A from the config.
4. User saddles again—no reset occurs for Token A (it's not in the active config).
5. Token A is re-added to the config later.
6. User claims Token A rewards accrued from the initial period, despite not staking during the new reward period.

This undermines the time-based reward vesting logic.

Root Cause

The protocol assumes `depositRewards` is reset during `saddle()`, but this reset:

- Only applies to currently active reward tokens.
- Ignores tokens removed and re-added during unsaddle periods.
- Fails to account for debt changes between saddle-unsaddle cycles.

— RECOMMENDATION

Consider reset of `depositRewards[accountId]` fully during `unsaddle()` to prevent stale reward data from persisting across states or config changes.

— RESOLUTION

Status: Acknowledged.

HIGH **H-02**

ACKNOWLEDGED

Liquidated collateral is credited twice

CATEGORY Unexpected Behavior

LOCATION <https://github.com/Synthetixio/synthetix-v3/blob/0c32d0cfc8a8cc7f22d368383fa10c598a318311/protocol/synthetix/contracts/modules/core/LiquidationModule.sol#L153-L154>

REVIEW Main Review

DESCRIPTION

When `LiquidationModule.liquidateToTreasury()` is called, the treasury account is credited the liquidated collateral twice

```
// move the collateral
Account.load(treasuryAccountId).increaseAvailableCollateral(
collateralType,
liquidationData.collateralLiquidated
);
treasuryAccountEpoch.collateralAmounts.set(
// solhint-disable-next-line numcast/safe-cast
bytes32(uint256(treasuryAccountId)),
// solhint-disable-next-line numcast/safe-cast
treasuryAccountEpoch.collateralAmounts.get(bytes32(uint256(treasuryAccountId))) +
liquidationData.collateralLiquidated
);
```

The first part of the code deposits the collateral into the treasury account and the second one delegates it to the appropriate vault in the treasury pool. Normally, when delegations happen, the available collateral of the given user is [decreased](<https://github.com/Synthetixio/synthetix-v3/blob/0c32d0cfc8a8cc7f22d368383fa10c598a318311/protocol/synthetix/contracts/modules/core/VaultModule.sol#L393>), otherwise it can be used again. As we can see from the code above, this step is missed here and in result the treasury account receives twice the collateral. This can lead to serious backing problems for Synthetix.

RECOMMENDATION

Consider removing the `increaseAvailableCollateral` call to achieve correct collateral delegation to the pool.

H-02 Liquidated collateral is credited twice

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

burnUsd locks the available collateral

CATEGORY	Rewards
LOCATION	IssueUSDModule.sol
REVIEW	Main Review

DESCRIPTION

A new check has been added to `burnUsd()` to make sure the locked collateral cannot be burned.

```
if (
  account.collaterals[usdToken.getAddress()].getTotalLocked() >
  account.collaterals[usdToken.getAddress()].amountAvailableForDelegationD18
) {
  revert InsufficientAvailableCollateral(
    account.collaterals[usdToken.getAddress()].amountAvailableForDelegationD18,
    account.collaterals[usdToken.getAddress()].getTotalLocked()
  );
}
```

The burn transaction will revert if we leave our available amount lower than what's locked. However, if we look at `[CollateralModule.withdraw()]`(<https://github.com/Synthetixio/synthetix-v3/blob/0c32d0cfc8a8cc7f22d368383fa10c598a318311/protocol/synthetix/contracts/modules/core/CollateralModule.sol#L92-L99>), we can see the locked amount is applied on the total user collateral amount, i.e available + already delegated. Because of this, the `burnUsd` will be reverting in cases where it should succeed. For example:

- `totalDeposited = 3000`; `assigned = 1000`; `locked = 800`; `availableForDelegation = 2000`
- The `unavailableCollateral = 1000`, because `1000 (assigned) > 800 (locked)`
- Therefore, we should be able to use all of the `availableForDelegation = 2000` collateral
- However, `burnUsd()` will require at least 800 to be left in `availableForDelegation`, which makes it a total of `1800` locked amount instead of `1000`.

This can result in the user being liquidated in situations where they need to repay their debt by burning sUsd and not being able to undelegate their collateral.

M-01 burnUsd locks the available collateral

CONTINUED

— RECOMMENDATION

Change the check in `burnUsd` to calculate `unavailableCollateral` taking into account the total amount.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Reward token removal can be abused for potential future earnings

CATEGORY Rewards
LOCATION TreasuryMarket.sol
REVIEW Main Review

DESCRIPTION

The owner can remove reward configurations from `depositRewardConfigurations`. This can be abused in the following way:

- A user frontruns the owner's transaction to `saddle` a large amount of funds which will record a big reward in their `depositRewards` for all the tokens (including the one to be removed)
- The owner transaction is executed and the given token is removed from `depositRewardConfigurations`
- After the minimum delegation time expires, the user can `unsaddle`. The removed token is not inside `depositRewardConfigurations`, the rest of it is the same as when the user saddled. Because of this, the user receives their fair share of rewards for the rest of the tokens, but their data in `depositRewards` for the removed token is not changed, so it continues accruing rewards.
- If that token is ever re-added, the user can frontrun the admin transaction again to saddle and receive their rewards even though they aren't eligible for it.

RECOMMENDATION

You can introduce a new array containing the active reward loans for each users and use it in the `unsaddle()` flow, by either paying the user the tokens or clearing the removed ones.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Delegation in `liquidateToTreasury` lacks checks

CATEGORY	Validation
LOCATION	LiquidationModule.sol
REVIEW	Main Review

— DESCRIPTION

When the liquidated collateral is delegated by adding it to `treasuryAccountEpoch.collateralAmounts`, `liquidateToTreasury()` the checks which usually run in `delegateCollateral` are not executed. This can lead to unexpected behaviors, including, but not limited to:

- delegating not enabled collateral if the liquidated collateral is not enabled on the treasury pool
- exceeding the pool collateral limit
- performing insufficient delegation, i.e below the minimum limit

— RECOMMENDATION

If the team considers these behaviours as acceptable, then document them. Otherwise, checks can be added for each desired validation, but keep in mind that each check added increases the probability of DOS for `liquidateToTreasury()`.

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Anyone can DoS setDepositRewardConfigurations

CATEGORY Informational

LOCATION <https://github.com/Synthetixio/synthetix-v3/blob/0c32d0cfc8a8cc7f22d368383fa10c598a318311/markets/treasury-market/contracts/TreasuryMarket.sol#L536-L537>

REVIEW Main Review

DESCRIPTION

Since `setDepositRewardConfigurations` reverts if `availableDepositRewards[token] > 0`, anyone can front-run it by calling `fundForDepositReward` with even 1 wei, causing a revert and blocking config updates.

[Code Reference](<https://github.com/Synthetixio/synthetix-v3/blob/0c32d0cfc8a8cc7f22d368383fa10c598a318311/markets/treasury-market/contracts/TreasuryMarket.sol#L492-L493>)

RECOMMENDATION

- Make `fundForDepositReward` permissioned
- or
- Call SNX Core to set `maxDepositable = 0` temporarily to allow removal.

[Code Reference](<https://github.com/Synthetixio/synthetix-v3/blob/0c32d0cfc8a8cc7f22d368383fa10c598a318311/protocol/synthetix/contracts/modules/core/MarketCollateralModule.sol#L46-L47>)

- or
- Add a function in the peripheral contract that calls `removeFromDepositReward(token, market.availableDepositRewards(token))` to remove the entire balance before making the call to set.

- or
- Use Front-running resistant RPC

RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Unsaddle may be blocked because of rewards

CATEGORY DoS
LOCATION TreasuryMarket.sol
REVIEW Main Review

DESCRIPTION

When a user unsaddles, any available rewards will be unconditionally claimed. However, if the rewards are still not deposited and the amount recored in `availableDepositRewards` is less than what's to be claimed, the unsaddle will revert. The reward mechanism serve the user as a benefit, but in this case it blocks their withdrawal which may create big inconvenience if the user needs the funds immediately.

RECOMMENDATION

Consider adding `forceUnsaddle` boolean parameter to the `unsaddle()` function which if true skips any rewards that cannot be paid out.

RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

The oracleManager should properly adjust decimals

CATEGORY Math
LOCATION TreasuryMarket.sol
REVIEW Main Review

DESCRIPTION

The `rewardAmount` computed in `saddle()` depends on the result returned by the `oracleManager` to convert the collateral token to reward token. The result of `rewardAmount` should be in `rewardTokenDecimals`. Currently, the calculation for it will result in `collateralTokenDecimals + oracleDecimals - 18`.

We should have `collateralTokenDecimals + oracleDecimals - 18 = rewardDecimals =>`
The following should hold true for each oracle:

```
oracleDecimals = rewardDecimals - collateralTokenDecimals + 18
```

RECOMMENDATION

Make sure each oracle returns the price in the correct decimal precision.

RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Unclaimed rewards are likely lost when DepositRewardToken is removed

CATEGORY Rewards
LOCATION TreasuryMarket.sol
REVIEW Main Review

— DESCRIPTION

The owner can remove reward configurations from `depositRewardConfigurations`. Because of this, any unclaimed rewards for users will be lost when they `unsaddle` since the loop won't execute any logic for the corresponding token.

— RECOMMENDATION

Make sure that in the event of reward token removal users are informed in advance so they can take the appropriate actions on time.

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Precision Revert in `withdrawMarketCollateral` During Unsaddle

CATEGORY	Logical Error
LOCATION	https://github.com/Synthetixio/synthetix-v3/blob/0c32d0cfc8a8cc7f22d368383fa10c598a318311/markets/treasury-market/contracts/TreasuryMarket.sol#L338
REVIEW	Main Review

— DESCRIPTION

The rewards allocated to users are deposited into the market as collateral by the treasury. These rewards are withdrawn when users claim them during the unsaddle process.

The `withdrawMarketCollateral` function internally calls `convertTokenToSystemAmount` to convert the token amount into the system's internal accounting units.

```
function convertTokenToSystemAmount(
    Data storage self,
    uint256 tokenAmount
) internal view returns (uint256 amountD18) {
    -----
    // ensure no precision is lost when converting to 18 decimals
    if (tokenAmount % (10 ** (decimals - 18)) != 0) {
        revert PrecisionLost(tokenAmount, decimals);
    }
    -----
}
```

However, if the token in question has more than 18 decimals, this conversion may revert due to precision loss during the conversion.

This could cause unexpected reverts during user unsaddles.

— RECOMMENDATION

If tokens with more than 18 decimals are used, ensure to truncate any precision beyond 18 decimals before invoking `withdrawMarketCollateral`.

— RESOLUTION

Status: Acknowledged.

LOW **L-07**

ACKNOWLEDGED

Approvals are not reset

CATEGORY	Logical Error
LOCATION	TreasuryMarket.sol
REVIEW	Main Review

— DESCRIPTION

The approvals for the reward tokens in the `TreasuryMarket` are never revoked, which means that if a token like USDT is used, its configuration cannot be edited and if once removed, it cannot be added again because the approval will try to change the already non-zero allowance value to another non-zero allowance value, which is not supported by these weird tokens.

— RECOMMENDATION

Consider using an approach like `safeApproveWithRetry()` which will reset the approval to 0 first if needed.

— RESOLUTION

Status: Acknowledged.

LOW **L-08**

ACKNOWLEDGED

The treasuryAccountId is not checked for existence when liquidating

CATEGORY Validation
LOCATION LiquidationModule.sol
REVIEW Main Review

DESCRIPTION

In `liquidateToTreasury()` the `treasuryAccountId` is fetched from the `_CONFIG_TREASURY_ACCOUNT_ID` config and is checked to not be 0, but there is no check to ensure the account exists.

RECOMMENDATION

Check for account existence.

```
uint128 treasuryAccountId = uint128(Config.readUint(_CONFIG_TREASURY_ACCOUNT_ID, 0));  
if (treasuryAccountId == 0) {  
    revert ParameterError.InvalidParameter("treasuryAccountId", "not set");  
}  
+ Account.exists(treasuryAccountId);
```

RESOLUTION

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

The pool loaded in `liquidateToTreasury` is not checked for existence

CATEGORY Validation
LOCATION LiquidationModule.sol
REVIEW Main Review

DESCRIPTION

In `liquidateToTreasury()` the `treasuryPoolId` is fetched from the `_CONFIG_TREASURY_POOL_ID` config and is checked to not be 0, but there is no check to ensure the pool exists.

RECOMMENDATION

When loading the pool, instead of `Pool.load()`, use `Pool.loadExisting()`. This will revert if the pool doesn't exist.

RESOLUTION

Status: Acknowledged.

LOW **L-10**

ACKNOWLEDGED

`_decimalPow()` supports only 18 decimals precision base

CATEGORY Math
LOCATION TreasuryMarket.sol
REVIEW Main Review

— DESCRIPTION

The `_decimalPow()` helper function in the `TreasuryMarket` contract raises a given `base` to an `exp` by multiplying `base` by itself `exp` times. However, it uses `mulDecimal` for the multiplication, which works by multiplying the two numbers and removing 18 decimals after that. After the first `cur = cur.mulDecimal(base)` is executed, the new `cur` will be in `base` decimals. This will result in a wrong result from the second iteration and return a wrong result. Currently, the code uses this function only in `_loanedAmount()` and the `base` there has 18 decimals of precision, but if you decide to use the function with a different base in the future, it can result in unexpected behaviors.

— RECOMMENDATION

Never use the function as it is for any `base` which is not in 18 decimals precision.

— RESOLUTION

Status: Acknowledged.

LOW **L-11**

ACKNOWLEDGED

Misplaced Comment in LiquidationModule.sol

CATEGORY Best Practices

LOCATION <https://github.com/Synthetixio/synthetix-v3/blob/0c32d0cfc8a8cc7f22d368383fa10c598a318311/protocol/synthetix/contracts/modules/core/LiquidationModule.sol#L239-L240>

REVIEW Main Review

— DESCRIPTION

The comment on [L239](<https://github.com/Synthetixio/synthetix-v3/blob/0c32d0cfc8a8cc7f22d368383fa10c598a318311/protocol/synthetix/contracts/modules/core/LiquidationModule.sol#L239-L240>) appears misplaced. It likely refers to logic on [L232](<https://github.com/Synthetixio/synthetix-v3/blob/0c32d0cfc8a8cc7f22d368383fa10c598a318311/protocol/synthetix/contracts/modules/core/LiquidationModule.sol#L232>).

— RECOMMENDATION

Consider moving the comment to the appropriate line (L232) or removing it if no longer relevant.

— RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.